

ZIG Astronomical AR Accuracy

Research and Architecture Blueprint for Smartphone-Based Star Alignment,
Plate Solving, and Celestial AR

Research prepared for the ZIG Android astronomy app

September 2026

Contents

1	1. Executive Conclusion	2
2	2. The Current Problem Model	3
3	3. State of the Art in Star Identification and Plate Solving	3
3.1	3.1 Astrometry.net — blind geometric hashing	3
3.2	3.2 ASTAP — local hash-based solving	4
3.3	3.3 The Pyramid algorithm — the star-tracker reference design	4
3.4	3.4 Tetra / Tetra3 — the practical open-source reference implementation	4
3.5	3.5 Triangle, polygon, pair-voting, geometric-voting, and k-vector meth- ods generally	5
3.6	3.6 Brightness-assisted and machine-learning approaches	5
4	4. Algorithm Comparison	5
5	5. Recommended Algorithm	6
6	6. Recommended Catalog	7
7	7. Recommended Camera Pipeline	7
8	8. Recommended Attitude Solution	8
9	9. Recommended Sensor Fusion	8
10	10. Recommended Tracking Architecture	8
11	11. Camera Calibration Architecture	9
12	12. Distortion Architecture	9
13	13. Timestamp Architecture	10
14	14. Failure Handling	10

15	15. Object-Class-Specific Accuracy Requirements	10
16	16. Hero Sky / Non-Camera Canvas Math	11
17	17. Android Implementation Split	11
18	18. Performance Estimate	12
19	19. Accuracy Estimate	13
20	20. Error Budget	13
21	21. Testing Methodology	15
22	22. Implementation Roadmap	15
23	23. Risk Register	15
24	24. What Not to Build, and Why	16
25	25. Final Architecture Diagram	16
26	26. Sources	17

1 1. Executive Conclusion

ZIG should adopt a **hybrid star-tracker architecture** (Architecture D in the comparison below): a lightweight, geometric-hashing star identification algorithm in the tradition of the Pyramid and Tetra3 solvers establishes an absolute celestial “lock” from the camera image on demand; between locks, the phone’s gyroscope carries orientation forward at high frequency; the magnetometer is demoted to a coarse fallback rather than a primary heading source; and ARCore is not used at all for celestial orientation, because its visual-inertial tracking depends on textured surroundings that a night sky does not provide.

This is not a novel invention — it is the same architecture used by real spacecraft star trackers (Pyramid, TRIAD/QUEST-based attitude solvers), scaled down using the same simplification that CubeSat-class open-source solvers such as ESA’s Tetra3 already apply: small pattern catalogs, geometric-hash lookup, and lost-in-space matching that assumes nothing about prior pointing.

The honest accuracy ceiling on a handheld consumer phone is **sub-degree to a few arcminutes immediately after a fresh lock**, degrading toward roughly a degree within tens of seconds of gyro-only tracking. Arcsecond-class, professional star-tracker accuracy (Level 6) is not achievable on this hardware and should not be promised to users — handheld tremor alone exceeds it. What ZIG *can* realistically deliver is something no existing consumer astronomy app does automatically: today, SkySafari and Star Walk ask the *user* to manually drag the sky until it lines up with a bright star or the Moon, every time they open the app, because their compass is known to be unreliable. ZIG can automate that exact correction with a few hundred

milliseconds of on-device plate solving instead of asking the user to do it by hand. That is the concrete, differentiating win.

2 2. The Current Problem Model

Correct ephemeris math (accurate RA/Dec, alt/az, observer location and time) is necessary but not sufficient for correct AR placement. Between “ZIG knows where Jupiter is” and “Jupiter’s dot lands on the real Jupiter on screen” sits an independent chain of transforms: device orientation sensing, gyro/accelerometer/magnetometer fusion, camera-to-device coordinate mapping, camera intrinsics (focal length, principal point), lens distortion, timestamp alignment between the camera frame and the orientation sample used to render it, and the perspective projection onto the screen. Every one of these is a separate source of angular error, and on a phone several of them are considerably larger than the astronomy engine’s own error. Improving the ephemeris math further, once it is already correct to arcsecond level, does nothing for AR accuracy — the bottleneck is entirely in the sensing/geometry chain described in this report.

A second, orthogonal distinction matters throughout: **lost-in-space** operation (the phone has no idea which way it is pointed and must solve for it from the image alone) versus **tracking** operation (the phone already knows its orientation approximately and only needs to maintain and refine it). These are different problems with different appropriate algorithms, and conflating them is a common design mistake.

3 3. State of the Art in Star Identification and Plate Solving

3.1 3.1 Astrometry.net — blind geometric hashing

Astrometry.net solves the general “no idea where this image is pointed” problem by extracting stars from an image, forming groups of four (“quads”), and computing a geometric hash from the quad’s shape — a set of coordinates invariant to translation, rotation, and scale. That hash is looked up in a pre-built index derived from a reference catalog (originally USNO-B, later augmented with 2MASS and Gaia-based indexes); a match becomes a hypothesized World Coordinate System solution, which is accepted only after passing a Bayesian verification test against a null hypothesis. Published results show above 99.9% success with effectively no false positives on typical survey imagery.

This is the most thoroughly validated approach in the field, but its design target is arbitrary deep-sky astrophotography — hundreds of detected stars, large multi-gigabyte multi-scale indexes covering the whole sky at many magnitude depths, and solve times measured in seconds. That is excessive for ZIG: a phone camera pointed at the night sky will detect a few dozen stars at most, not hundreds, and does not need whole-sky, all-magnitude coverage.

3.2 3.2 ASTAP — local hash-based solving

ASTAP follows the same conceptual approach — brightest-star selection, geometric hash of the four closest stars (“constellation”), comparison against a precomputed catalog hash table — but is designed as a **local** solver: it expects an approximate starting pointing and field of view rather than solving blind across the whole sky, which makes it faster and lighter than Astrometry.net. This is architecturally closer to what ZIG needs once it has any prior estimate of pointing (e.g., from the astronomy engine’s last known alt/az plus gyro tracking), though ZIG still needs a true lost-in-space capability for the very first lock of a session, when the app cannot yet trust the phone’s orientation sensors at all.

3.3 3.3 The Pyramid algorithm — the star-tracker reference design

Developed by Mortari, Samaan, Bruccoleri and Junkins and flown operationally on spacecraft including the HETE satellite and Draper Laboratory’s Inertial Stellar Compass, Pyramid is the most directly relevant precedent because it was designed for exactly ZIG’s constraint profile: no prior pointing knowledge, a small number of visible stars, and a real possibility of false detections. It builds on triangle-based matching but adds a fourth star to form a “pyramid,” which is what gives it its central property: a triangle match can occur by coincidence, but a *consistent* four-star match essentially cannot, so Pyramid tolerates a large number of spurious detections (the original paper demonstrates robust identification with as few as four true stars and up to 24 false ones) while still running in single-digit milliseconds to under a second depending on implementation. It relies on the **k-vector range-searching technique** (also Mortari’s) to look up candidate catalog star pairs by measured angular separation in near-constant time, independent of catalog size, rather than doing a linear or tree-based search.

3.4 3.4 Tetra / Tetra3 — the practical open-source reference implementation

Tetra3, released by the European Space Agency (Apache 2.0 license, Python, with community forks), is a direct, working demonstration that this class of algorithm runs comfortably on modest hardware. It implements a geometric-hashing pipeline conceptually descended from Pyramid/Astrometry.net-style quad hashing: four bright centroids are combined into six pairwise angular separations, normalized into five edge ratios that are invariant to pointing and rotation, and quantized into a hash key against a precomputed table. Documented performance is roughly **10 milliseconds per solve** (after centroids are extracted) at about **10 arcseconds** of solved accuracy, using a database built from the Yale Bright Star Catalog (about 9,100 stars, complete to magnitude 7) or the larger Hipparcos/Tycho catalogs for narrower fields of view. This is the single most useful concrete data point in this whole investigation: it is a real, licensed, working lost-in-space solver whose resource profile is compatible with a phone, and its architecture (small pattern database, geometric hash, k-vector-style lookup) is what ZIG should adapt rather than re-deriving from first principles or porting the much heavier Astrometry.net.

3.5 3.5 Triangle, polygon, pair-voting, geometric-voting, and k-vector methods generally

These are variations on the same underlying idea — encode small star groupings as scale/rotation-invariant geometric descriptors and match them against a precomputed catalog index — differing mainly in how many stars form the base unit (pairs, triangles, quads/pyramids), how the index is searched (hash table, k-vector, k-d tree), and how false matches are rejected. The consistent finding across this literature, echoed by Pyramid’s own design rationale, is that going from three-star (triangle) to four-star (pyramid/quad) matching sharply reduces the false-positive rate for a modest increase in computation, which is why virtually every modern practical solver — Astrometry.net, ASTAP, Tetra3 — converged on four-star quads rather than triangles. There is no good reason for ZIG to use anything smaller.

3.6 3.6 Brightness-assisted and machine-learning approaches

Star brightness as measured by a smartphone camera is not a reliable primary identification feature: automatic exposure, HDR merging, variable gain, white balance, atmospheric extinction, and sensor saturation all distort measured magnitude independently of the star’s true brightness, and none of that is controllable the way it is on a dedicated astronomical camera. Every serious star-identification algorithm surveyed here — Pyramid included — uses angular geometry as the primary matching key and treats brightness, when used at all, as a secondary tie-breaker or verification signal, never the primary key. Neural-network star identification exists in the research literature, but it solves the same lost-in-space problem the geometric methods already solve well, at the cost of a model to ship, maintain, and validate offline; there is no evidence in the literature that it outperforms geometric hashing enough on this problem to justify that cost for ZIG.

4 4. Algorithm Comparison

Approach	Basis	Phone feasibility	Notes
Astrometry.net	Quad geometric hashing, whole-sky index	Theoretically superior but impractical as-is	Multi-GB indexes, seconds-scale solves; built for arbitrary deep-sky images, not phone AR
ASTAP	4-star hash, needs approximate prior pointing	Practical, but only as a <i>secondary</i> re-lock solver	Not a true lost-in-space solver; good complement once ZIG has any pointing prior

Approach	Basis	Phone feasibility	Notes
Pyramid	4-star “pyramid,” k-vector lookup	Highly practical	Reference design for exactly this problem; flight-proven; tolerant of false detections
Tetra3	Quad geometric hashing (Pyra- mid/Astrometry.net lineage), small DB	Highly practical	Real open-source implementation at phone-friendly scale; ~10 ms/solve, ~10” accuracy documented
Triangle / pair matching	3-star or 2-star geometric descriptors	Practical but inferior	Higher false-positive rate than 4-star methods for a given star count
Geometric voting	Pairs “vote” for catalog stars	Potentially practical	Robust to noise, but no phone-scale reference implementation found
Brightness- assisted ID	Magnitude + geometry	Impractical as a primary key	Smartphone- camera expo- sure/HDR/gain make brightness unreliable; usable only as a secondary check
ML / neural star-ID	Learned features	Potentially practical, not justified	No demonstrated phone-relevant advantage over geometric hashing; adds model/maintenance cost

5. Recommended Algorithm

A Tetra3-style geometric-hashing quad solver, extended with Pyramid’s four-star verification discipline and Mortari’s k-vector-style angular lookup for the catalog index. Concretely: detect star centroids, form 4-star combinations from the brightest available stars, compute the six pairwise angular separations and reduce

them to rotation/scale-invariant edge ratios, hash-lookup candidate catalog quads, and accept a match only when it is internally consistent across all six pairwise distances within the expected centroiding tolerance (Pyramid’s core robustness trick). This gives ZIG a lost-in-space capability without inheriting Astrometry.net’s whole-sky index size or multi-second solve times, while still directly benefiting from that lineage of validated mathematics rather than inventing new geometry from scratch.

6 6. Recommended Catalog

Use the **Yale Bright Star Catalog class of data (roughly 9,000-9,500 stars, complete to about magnitude 7)** as the default on-device index, with an optional deeper mode built from **Hipparcos (~118,000 stars, ~1 mas positional accuracy)** for narrower fields of view or darker-sky users. **Do not use Tycho-2 (2.5 million stars) or Gaia (over a billion sources) as the on-device plate-solving catalog** — their extra depth targets stars far fainter than a handheld phone camera can realistically detect in a single unstabilized frame, and their size is disproportionate to offline Android storage and lookup-speed constraints. This recommendation is reinforced by a practical constraint the original brief did not fully account for: a handheld phone, without a tripod, cannot take a multi-second tracked exposure without motion blur, which means the *effective* limiting magnitude achievable in practice is closer to naked-eye bright stars (roughly magnitude 3-5) than to the magnitude-7-to-11 depth these bigger catalogs target. A small, bright-star catalog is not a compromise here — it is the correct-sized catalog for what the camera can actually see.

Gaia-derived positions are worth using for a different purpose: as the *source data* from which ZIG’s bright-star subset is extracted and periodically refreshed, since Gaia is now the most accurate positional reference available, even though the full Gaia catalog itself should never ship on-device.

7 7. Recommended Camera Pipeline

Star detection should follow the same front end used by every solver surveyed here: background estimation and adaptive thresholding, connected-component blob detection on the residual, sub-pixel centroiding via a weighted (intensity-weighted center-of-gravity or Gaussian) fit rather than integer pixel peaks, and explicit handling of saturated cores (bright stars commonly blow out several pixels on a phone sensor, which biases a naive center-of-gravity fit if not corrected). Work on the sensor’s demosaiced but otherwise minimally processed frame where possible — the more the platform’s automatic HDR/multi-frame merge pipeline touches the image before ZIG sees it, the less trustworthy both position and brightness become. Motion blur and rolling-shutter smear should be flagged (e.g., via elongated-blob detection) and either corrected for centroiding or used to reject that frame for solving, rather than silently accepted.

8 8. Recommended Attitude Solution

This is a direct instance of **Wahba’s problem**: given several star directions measured in the camera frame and the same stars’ known directions in the catalog (celestial) frame, find the rotation that best aligns them. **QUEST** is the right default solver: it is the most widely used, well-validated closed-form solution to Wahba’s problem for two or more observations, strikes a good balance of speed and robustness, and has decades of spacecraft flight heritage. TRIAD remains useful as a fast two-star fallback when a full multi-star solve is not available (e.g., only two stars visible). Before the attitude solve, candidate star matches should pass through a **RANSAC-style outlier rejection** step, since a single misidentified star can otherwise corrupt an otherwise-good solution — this is standard practice and is what lets the pipeline tolerate the occasional false match that four-star verification doesn’t catch.

9 9. Recommended Sensor Fusion

Do not use ARCore as the primary orientation source for celestial AR. ARCore’s motion tracking is a visual-inertial system that depends on detecting and following textured feature points in the surrounding scene; documented behavior and independent benchmarking both show it degrading or losing tracking in low-light, low-texture, and outdoor/dynamic environments — and a night sky, from the camera’s point of view, is close to the worst case of “no usable visual features” that this technology was built to handle. Building ZIG’s celestial AR on top of ARCore’s world tracking would be building on a foundation designed for a different problem.

Instead, use Android’s fused **rotation-vector sensor** (gyroscope-driven, accelerometer- and magnetometer-assisted orientation) as the continuous tracking backbone, and **explicitly down-weight or disable the magnetometer’s contribution once a star-based lock exists**. This mirrors what today’s best consumer astronomy apps already tell their own users: SkySafari’s documentation states plainly that sky-chart misalignment is expected because of “the inaccuracy inherent in your device’s digital compass,” and both SkySafari and Star Walk’s support material walk users through manually dragging the chart to align it with a known bright object — magnetic-north-based heading on a phone is simply not trustworthy near buildings, vehicles, speakers, or magnetic phone cases. A star-derived absolute orientation is categorically better than magnetometer heading whenever one is available, and should supersede it.

10 10. Recommended Tracking Architecture

Architecture D (Hybrid Star Tracker), not Architecture C (repeated full plate solving) and not Architecture A/B (sensor-only or sensor-plus-occasional-correction) alone:

1. On session start (or whenever confidence is lost), run a full lost-in-space solve to establish an absolute lock.

2. Between locks, integrate gyroscope data at high frequency for smooth, low-latency tracking — this is cheap and is what makes the AR view feel responsive.
3. Periodically (not every frame — see performance discussion below) attempt a lightweight re-lock using the *current* pointing as a prior, which is exactly the regime ASTAP-style local solving is good at, cheaper than a fresh lost-in-space solve.
4. On sustained disagreement between gyro-predicted and star-observed position, trust the stars and re-anchor; on repeated re-lock failure (clouds, city glare, camera pointed at buildings), fall back gracefully (Section 14).

Running full plate-solving continuously at 30 fps is neither necessary nor a good use of a phone’s thermal and battery budget: gyro integration is accurate over the few-hundred-millisecond-to-few-second intervals between re-locks, and the re-lock’s job is only to correct the small drift that accumulates in that window, not to re-discover orientation from nothing every frame.

11 11. Camera Calibration Architecture

Android’s Camera2 API exposes LENS_INTRINSIC_CALIBRATION (focal lengths, principal point) and, from API 28, LENS_DISTORTION directly as device metadata — when present, this is the cheapest and most direct source of camera intrinsics and should be queried first. **Critically, both fields are explicitly optional and may be null on a given device**, so ZIG cannot depend on them being available. The recommended fallback, and arguably the more elegant long-term solution, is **self-calibration as a byproduct of plate solving itself**: once several catalog stars are matched to detected image positions, the same least-squares machinery used in professional plate solving (fitting a WCS solution, optionally with a SIP-style polynomial distortion term) can solve for the camera’s effective focal length, principal point, and radial distortion directly from the star field, with no checkerboard, no per-device calibration step, and no dependency on metadata a given phone may not report. This should be refined opportunistically over multiple successful locks per device/lens combination and cached, rather than solved fresh (and discarded) every session.

12 12. Distortion Architecture

Recommend a **hybrid split**, not full-frame undistortion: undistort only the detected **star centroid coordinates** before they enter quad-matching and the attitude solve (cheap — a handful of points, not a full-resolution image), and apply the **forward distortion model to the rendered AR overlay** so it lines up with the distorted camera preview the user actually sees on screen. Undistorting the entire live camera frame every frame is unnecessary computational cost for a problem that only needs a few dozen point coordinates corrected, and it also avoids introducing resampling artifacts into the preview image itself.

13 13. Timestamp Architecture

Use the `Camera2.CaptureResult.SENSOR_TIMESTAMP` for the frame that was actually solved, and interpolate (quaternion `slerp`, not linear) the gyro-integrated orientation history to that exact instant, rather than using “whatever the latest sensor reading happens to be” at the time the solve result comes back. **This matters more than it looks:** a rotation rate of roughly $45^\circ/\text{second}$ — an entirely ordinary handheld panning speed — combined with a mere 20-millisecond mismatch between the camera exposure instant and the orientation sample used to render it produces on the order of **0.9° of angular error** on its own, which is comparable to or larger than several other error sources in this report combined. Phones are not guaranteed to share a single clock domain across camera, gyroscope, and ARCore/VIO subsystems out of the box, so this synchronization needs to be verified per platform rather than assumed.

14 14. Failure Handling

A graceful degradation ladder, not a binary works/fails behavior:

- **Full lock (4+ verified stars):** normal operation, highest confidence.
- **Marginal lock (2-3 stars, or low geometric diversity):** accept a lower-confidence TRIAD/constrained solve, but flag reduced confidence internally and bias toward requesting a re-lock soon.
- **No usable stars (clouds, urban light pollution, camera pointed at buildings/trees, saturation, heavy motion blur):** fall back to the last known good orientation plus gyro-only dead reckoning, with confidence decaying over time, and surface a lightweight UI cue inviting the user to point at open, darker sky — functionally the automated equivalent of the manual “drag to align on a bright object” step SkySafari and Star Walk already ask users to do today.
- **Ambiguous or conflicting matches:** discard rather than guess; a wrong confident lock is worse than an honest “not locked” state.

15 15. Object-Class-Specific Accuracy Requirements

Not every object needs the same correction budget. Stars are effectively fixed points for AR purposes (proper motion over even 50 years is at most a few arcminutes for a handful of unusually fast-moving stars, and negligible — well under an arcsecond to a few arcseconds — for the overwhelming majority, far below the phone’s own multi-arcminute-to-degree error floor, so Gaia-style epoch propagation is not worth the complexity). The Sun, Moon, and planets need topocentric correction (parallax matters most for the Moon, at up to roughly a degree depending on geometry) and atmospheric refraction near the horizon; refraction is well modeled by Bennett’s formula and is negligible above about 45° altitude, roughly 18-25 arcminutes around $10\text{--}20^\circ$ altitude, and rises sharply to around half a degree — comparable to the Sun’s own apparent diameter — right at the horizon. Satellite and ISS positions come from an entirely separate propagation (SGP4/SDP4 on TLE data) and should not be touched by any of this work; the only thing that should change is that satellites, like everything else, get projected onto the screen through the same calibrated camera-attitude

pipeline once it exists, rather than through a separately-tuned transform.

16 16. Hero Sky / Non-Camera Canvas Math

The non-AR “Hero Sky” canvas should be driven by the **same vector-based azimuth/altitude projection math** as the camera AR view, not a separate ad hoc implementation — this is both simpler to maintain and guarantees the two views never disagree. The specific behavioral requirement in the brief (northern-hemisphere observer facing south: east/sunrise on the left, west/sunset on the right; southern-hemisphere observer facing north: mirrored) is worth stating precisely, because it is **not a special case requiring hemisphere-conditional code** — it is an automatic consequence of standard relative-bearing math. Using the ordinary convention (azimuth measured clockwise from north; screen-x increasing to the right), the correct general formula is:

```
relative_azimuth = wrap_to_180( object_azimuth - viewer_facing_azimuth )
screen_x = center_x + scale * relative_azimuth
```

Standing facing south (viewer_facing_azimuth = 180°) and evaluating due east (object_azimuth = 90°) gives relative_azimuth = -90°, which places it left of center — matching the requirement. Standing facing north (viewer_facing_azimuth = 0°) and evaluating the same due-east point gives relative_azimuth = +90°, placing it right of center — also matching the requirement, with no hemisphere check anywhere in the formula. **If ZIG’s current Hero Sky implementation contains an explicit hemisphere-based left/right flip for this behavior, that is worth auditing** — it likely either duplicates what the relative-bearing math already does correctly, or compensates for a sign error introduced somewhere else in the pipeline. Near the zenith this simple azimuth-difference approach degrades (azimuth becomes ill-defined directly overhead), so the underlying implementation should ultimately convert both the viewing direction and each object’s alt/az into unit vectors and project through a proper camera/perspective matrix — the same representation recommended for the AR attitude solution in Section 8 — rather than doing flat azimuth subtraction, with the formula above serving as the correct mental model and a useful test case.

17 17. Android Implementation Split

Layer	Recommended language/tooling	Rationale
Star detection, centroiding	C++/JNI	Per-pixel/per-frame hot path; benefits from native performance
Quad hashing, catalog lookup	C++/JNI (with a small precomputed binary index shipped as an asset)	Performance-critical, and matches the reference implementations surveyed (Tetra3’s own core is written for speed)

Layer	Recommended language/tooling	Rationale
Attitude solve (QUEST/TRIAD/RANSAC)	C++/JNI or Kotlin	Small matrices, low call frequency (once per lock, not per frame) — either is fine; keep in the same module as the rest of the solving pipeline for simplicity
Sensor fusion / gyro integration	Kotlin, using Android's SensorManager rotation-vector sensor	Standard platform API, no native code needed
AR overlay rendering, projection	GPU (Compose/Canvas or a lightweight GL layer)	Rendering is cheap once the attitude and projection are known; this is not where the accuracy problem lives
ARCore	Not used for celestial orientation	See Section 9
OpenCV	Optional, for camera calibration utilities and centroid/blob tooling only	Not required for the core solving pipeline if a purpose-built lightweight implementation (à la Tetra3) is used instead; adding the full OpenCV native dependency purely for star detection is heavier than necessary

18. Performance Estimate

Using Tetra3's own documented figures as the concrete reference point — roughly 10 ms per solve once centroids are extracted, at 10-arcsecond solved accuracy, on a database sized for a bright-star catalog — a phone-native (C++/JNI) implementation of an equivalent pipeline should comfortably solve in well under 100 ms end-to-end including centroid extraction, which is fast enough to run on demand (button press, periodic re-lock, or confidence-triggered) without a noticeable stall, and far too slow to need running at full camera frame rate — which, as noted in Section 10, it should not do anyway. Storage cost for a bright-star catalog and its precomputed quad-hash index is measured in single-digit megabytes, not the multi-gigabyte footprint of a whole-sky, deep-magnitude index. Battery/thermal cost is dominated by camera sensor activity during active AR use, which is a cost ZIG already pays for the existing AR sky view regardless of this work.

19 19. Accuracy Estimate

Level (per brief's scale)	Description	Realistically achievable?
1	Better than conventional compass-only AR	Yes — trivially, via calibrated sensor fusion alone
2	Good enough for casual/naked-eye astronomy	Yes — sensor fusion with distortion-corrected FOV, no plate solving strictly required
3	Very good amateur astronomy AR	Yes — with calibrated camera geometry and magnetometer de-weighted
4	Sub-degree precision	Yes — with periodic star-based re-lock (this report's core recommendation)
5	Few-arcminute-class, favorable conditions	Yes, but only briefly — right after a fresh lock, on a steadied phone, in genuinely dark sky; degrades within tens of seconds
6	Near-professional / spacecraft-star-tracker class	No. Handheld tremor, consumer lens quality, rolling shutter, and timestamp jitter form a hard floor well above arcsecond class. Do not promise this to users.

20 20. Error Budget

Values below are order-of-magnitude estimates for a typical modern Android phone under typical handheld night-sky conditions. Rows marked **[sourced]** come directly from material cited in Section 23; rows marked **[derived]** are straightforward physical/geometric calculations from those same figures; rows marked **[engineering estimate]** are reasoned approximations where no single authoritative phone-specific figure exists.

Error source	Typical magnitude	Basis
Magnetometer heading	2°-10°, worse (tens of degrees) near metal/magnets/speakers	[sourced] — SkySafari/Star Walk both document unreliable compass heading requiring manual correction
Gyro drift (unaided integration)	A few degrees per minute	[engineering estimate] — typical consumer MEMS gyro bias-stability behavior
Camera FOV/focal-length assumption error	~1.7° at frame edge for a 5% FOV error on a ~70° FOV frame	[derived]
Lens distortion (uncorrected, wide/ultrawide)	Up to several degrees at frame edges	[engineering estimate]
Star centroiding	~10-15 arcsec, for 0.2 px centroid error at typical smartphone plate scale (~60 arcsec/px)	[derived]
Plate-solving residual (Tetra3-class algorithm)	~10 arcsec	[sourced]
Timestamp mismatch	~0.9° for a 20 ms offset at a 45°/s pan rate	[derived]
Atmospheric refraction	Negligible above ~45° altitude; ~18-25' around 10-20° altitude; up to ~0.5° at the horizon	[sourced]
Location error (GPS-level, tens of meters)	Negligible for stars/planets; matters only for wrong-city-scale errors	[engineering estimate]
System clock error	~15 arcsec of hour-angle error per second of clock error	[derived — basic sidereal rate]
Astronomy engine (ephemeris)	Sub-arcsecond with full series (VSOP87/ELP2000); ~1' with simplified low-precision formulas	[engineering estimate, standard astronomical algorithm knowledge]
Rendering/projection	Negligible if done in double precision	[engineering estimate]

The dominant, addressable errors on a phone are magnetometer heading, camera calibration/distortion, and timestamp synchronization — not the astronomy engine. This is the central justification for this report's architecture: it targets exactly these three.

21 21. Testing Methodology

- **Static bench test:** phone fixed on a mount pointed at a known star field; compare predicted vs. actual screen position; report RMS, median, and 95th-percentile angular error.
- **Rotation test:** slow controlled rotation through a full 360° azimuth sweep and several elevation angles; verify no systematic bias appears at particular orientations (a sign of an uncorrected distortion or timestamp issue).
- **Dynamic test:** rapid handheld motion; measure re-lock latency and drift accumulated during gyro-only tracking.
- **Hemisphere test:** verify the Section 16 screen-orientation behavior explicitly in both hemispheres and near the equator.
- **Sky-condition test:** dark sky, suburban sky, urban sky, partial cloud — measure plate-solve success rate and graceful-degradation behavior (Section 14), not just accuracy when it works.
- **Device/lens test:** main, ultrawide, and telephoto lenses across a few representative phone models, since distortion and available LENS_INTRINSIC_CALIBRATION support vary by device.

Track concretely: RMS and 95th-percentile angular error, plate-solve success rate, false-identification rate, drift rate per minute during gyro-only tracking, and re-lock latency.

22 22. Implementation Roadmap

1. **Phase 1 — Calibrated sensor fusion baseline.** Rotation-vector-based tracking, magnetometer de-weighting, camera FOV/distortion correction using LENS_INTRINSIC_CALIBRATION/LENS_DISTORTION where available. This alone should reach Level 3.
2. **Phase 2 — Star detection and centroiding pipeline.** Build and validate the detection front end in isolation (feed it known star-field test images) before wiring it into a solver.
3. **Phase 3 — Lost-in-space quad solver.** Bright-star catalog, quad hashing, k-vector lookup, Pyramid-style 4-star verification, QUEST attitude solve with RANSAC outlier rejection.
4. **Phase 4 — Hybrid tracking loop.** Gyro tracking between locks, periodic lightweight re-lock, confidence-based fallback ladder (Section 14).
5. **Phase 5 — Self-calibration.** Opportunistic camera-intrinsics refinement from successful locks, cached per device/lens.
6. **Phase 6 — Validation and tuning.** Run the Section 21 test matrix across real devices and conditions; tune re-lock frequency and confidence thresholds against real-world battery/accuracy tradeoffs.

23 23. Risk Register

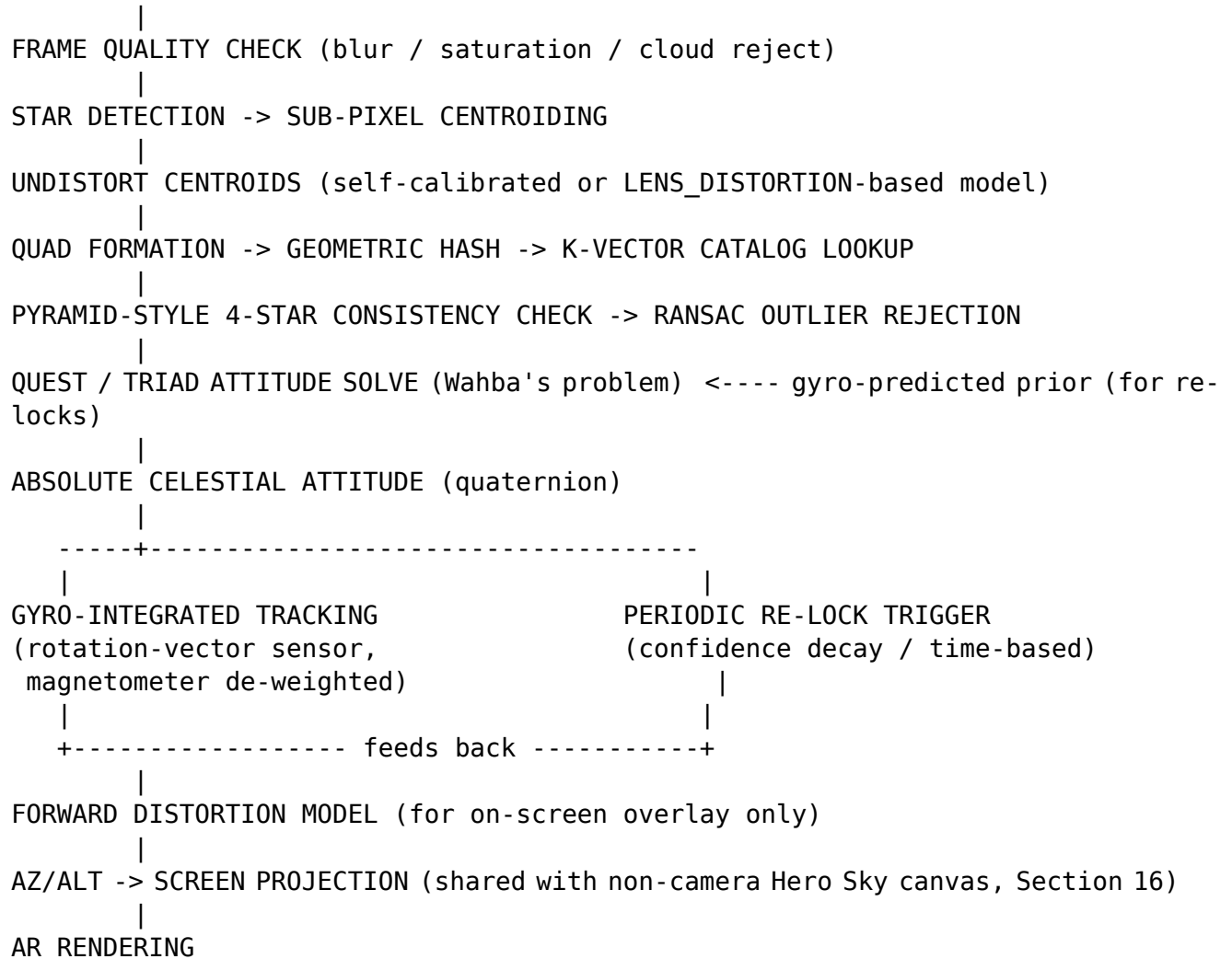
Risk	Impact	Mitigation
Too few visible stars in a phone-camera frame in light-polluted or hazy sky	Plate solving fails when it's needed most	Graceful fallback (Section 14); bright-star catalog maximizes odds with few detections
Camera/IMU clock domains not actually synchronized on a given device	Silent accuracy loss that looks like “the algorithm doesn’t work” rather than a sync bug	Explicit per-device timestamp verification in Phase 6, not an assumption
LENS_INTRINSIC_CALIBRATION or LENS_DISTORTION absent on many real devices	Online calibration path becomes the common path, not the exception	Design self-calibration (Section 11) as a first-class path from the start, not an afterthought
Chasing Level 6 accuracy	Wasted engineering effort against a hardware-imposed ceiling	Set and communicate Level 4–5 as the target explicitly (Section 19)
Continuous full-frame plate solving	Battery/thermal drain, no accuracy benefit over periodic re-lock	Architecture D’s periodic re-lock design (Section 10)

24 24. What Not to Build, and Why

- **Do not port or re-implement Astrometry.net’s full blind-solving architecture.** It targets a different problem (arbitrary deep-sky imagery, whole-sky coverage, many hundreds of detected stars) and its index size and solve times are wrong for a phone. Its mathematical lineage (quad geometric hashing) is exactly what to borrow — its scale is not.
- **Do not build celestial orientation on top of ARCore.** It is solving a different problem (textured-environment visual-inertial odometry) and a night sky is close to its worst case.
- **Do not use star brightness as a primary identification feature.** Smartphone camera exposure/HDR/gain behavior makes it unreliable, and no serious star-identification algorithm surveyed here relies on it as the primary key.
- **Do not run full plate-solving every frame.** Gyro tracking between periodic re-locks achieves the same practical accuracy at a fraction of the compute and battery cost.
- **Do not target or promise arcsecond-class (Level 6) accuracy.** It is not achievable on this hardware, and setting it as a goal will waste engineering time chasing a ceiling that handheld tremor alone already caps.
- **Do not apply Gaia-scale proper-motion/epoch propagation.** The correction is smaller than the phone’s own error floor for all but a handful of catalog stars.

25 25. Final Architecture Diagram

CAMERA FRAME (+ SENSOR_TIMESTAMP)



26 26. Sources

This report draws on, and paraphrases rather than quotes, the following:

- Lang, D. et al., *Astrometry.net: Blind Astrometric Calibration of Arbitrary Astronomical Images*, arXiv:0910.2233 / AJ 139:1782 (2010).
- ASTAP star pattern recognition and astrometric (plate) solving algorithm documentation, hnsky.org.
- Ignat, O., "How astronomic plate-solving works," oleignat.com (2021).
- Wikipedia, "Astrometric solving."
- ESA / GorgiAstro, *tetra3: A fast lost-in-space plate solver for star trackers*, GitHub repository and documentation (Apache 2.0).
- Mortari, D., Samaan, M. A., Bruccoleri, C., Junkins, J. L., "The Pyramid Star Identification Technique," *Navigation* 51(3), 2004.
- Mortari, D., Junkins, J. L., Samaan, M. A., "Lost-in-Space Pyramid Algorithm for Robust Star Pattern Recognition," AAS 01-004 (2001).
- Mortari, D., Neta, B., "k-Vector Range Searching Techniques," AAS 00-128

(2000).

- Markley, F. L., and related NASA NTRS technical reports on Wahba’s problem, QUEST, TRIAD, and Davenport’s q-method.
- Wikipedia, “Triad method”; Wikipedia, “Wahba’s problem.”
- “A Benchmark Comparison of Four Off-the-Shelf Proprietary Visual-Inertial Odometry Systems,” PMC9785098 (2022).
- Google Developers, ARCore motion-tracking guidance and Geospatial API documentation.
- “CloudAR: A Cloud-based Framework for Mobile Augmented Reality,” arXiv:1805.03060.
- SkySafari user guide, “Compass and AR Help” and “Compass & Gyro Help,” skysafariastronomy.com.
- Star Walk support, “Compass problem — how to calibrate the compass,” starwalk.space.
- Tycho-2 Catalogue documentation, NOIRLab Astro Data Lab and NASA HEASARC.
- Hipparcos/Tycho-2 star-count and accuracy figures as summarized in Gaia DR2 catalogue-validation documentation (ESA).
- British Astronomical Association, “Atmospheric Refraction” (2019).
- Asterism.org, “Atmospheric Extinction and Refraction.”
- celestialprogramming.com, approximate atmospheric refraction formulas (Bennett).
- OpenCV documentation and LearnOpenCV, “Camera Calibration using OpenCV” (Zhang’s method).
- Android Developers, CameraCharacteristics.LENS_INTRINSIC_CALIBRATION and LENS_DISTORTION reference documentation.

Prepared as a research and architecture blueprint. It does not reference or assume access to ZIG’s source code, and no implementation claims are made about ZIG’s current codebase — see Section 2 onward for architecture recommendations to be mapped onto the real codebase separately.